Model-Based RL

CS 185/285

Instructor: Sergey Levine
UC Berkeley

Part 1:
Can we learn a “simulator”?

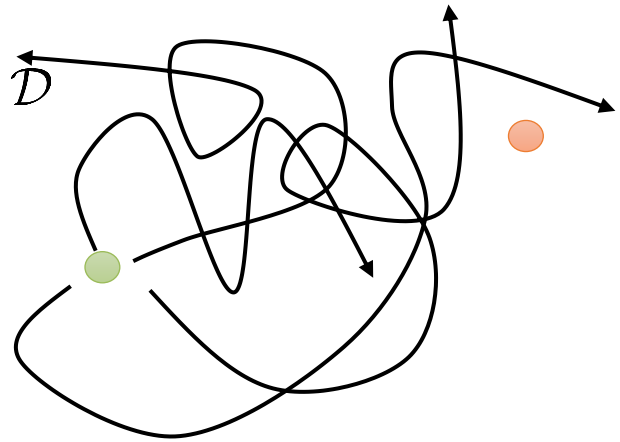
Can we learn a “simulator”?

Veo 2

Sora

Notional "learned simulator" algorithm

1. Get some data $\mathcal{D} = \{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i\}$
   collected using base policy $\pi_\beta(\mathbf{a}|\mathbf{s})$

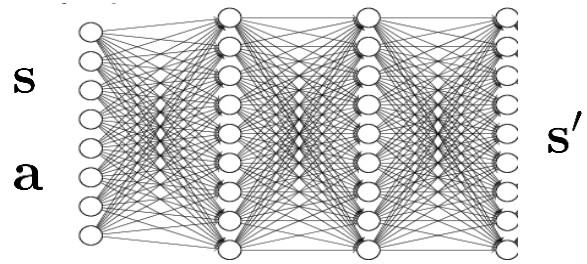


Will this work **well?**

2. Learn "simulator" $f(\mathbf{s}, \mathbf{a}) = \mathbf{s}'$

   $\min_f \sum_i ||f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i||^2$

Why might it **fail?**



3. Run your favorite RL method inside "simulator" f

For now it doesn't really matter how this works

- Literally run *model-free* RL in the "simulator"
- Use a planning or trajectory optimization algorithm

Notional "learned simulator" algorithm

Why might it fail?

1. Get some data $\mathcal{D} = \{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i\}$ collected using base policy $\pi_\beta(\mathbf{a}|\mathbf{s})$

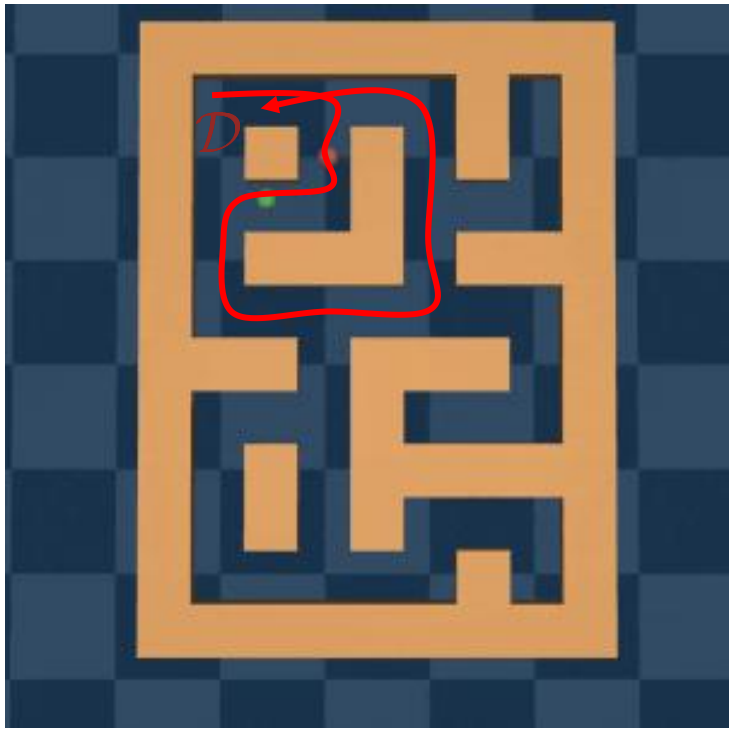
this policy matters a lot

yet another form of *distributional shift*

2. Learn "simulator" $f(\mathbf{s}, \mathbf{a}) = \mathbf{s}'$

$$\min_f \sum_i ||f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i||^2$$

3. Run your favorite RL method inside "simulator" f



Notional "learned simulator" algorithm

Why might it fail?

1. Get some data $\mathcal{D} = \{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i\}$ collected using base policy $\pi_\beta(\mathbf{a}|\mathbf{s})$

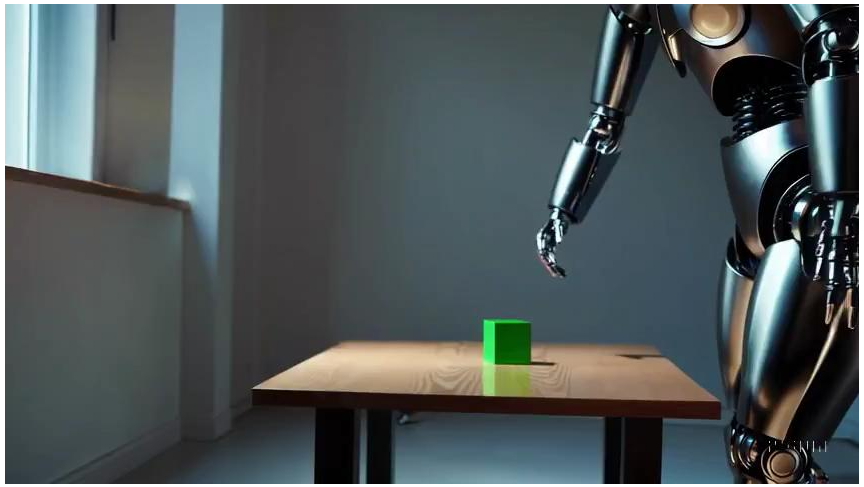
2. Learn "simulator" $f(\mathbf{s}, \mathbf{a}) = \mathbf{s}'$

$$\min_f \sum_i ||f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i||^2$$

3. Run your favorite RL method inside "simulator" f

In practice, the design of the model is *domain dependent*

Some domains are much easier to simulate than others



A humanoid robot standing near the table with red, green and blue cubes on it performs a cube stacking task, with red in the bottom and blue on the top.

Notional “learned simulator” algorithm

1. Get some data $\mathcal{D} = \{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i\}$ collected using base policy $\pi_\beta(\mathbf{a}|\mathbf{s})$

2. Learn “simulator” $f(\mathbf{s}, \mathbf{a}) = \mathbf{s}'$

   $$\min_f \sum_i ||f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i||^2$$

3. Run your favorite RL method inside “simulator” f

- Big neural models can be much more expensive computationally than “conventional” simulators
- Just because we have a “simulator” doesn’t mean that planning/RL is “easy”
- Planning and optimal control directly on image pixels is very difficult
- Model-free RL in the learned “simulator” might be very difficult (and time-consuming) too!
- Sometimes we might opt for a less accurate (e.g., simpler) model just to make planning/RL easier

Notional “learned simulator” algorithm

1. Get some data $\mathcal{D} = \{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i\}$ collected using base policy $\pi_\beta(\mathbf{a}|\mathbf{s})$

2. Learn “simulator” $f(\mathbf{s}, \mathbf{a}) = \mathbf{s}'$

   $\min_f \sum_i ||f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i||^2$

3. Run your favorite RL method inside “simulator” f

Statistics/algorithms problem

Fun to talk about in class, because we actually have a good understanding

Deep learning/models problem

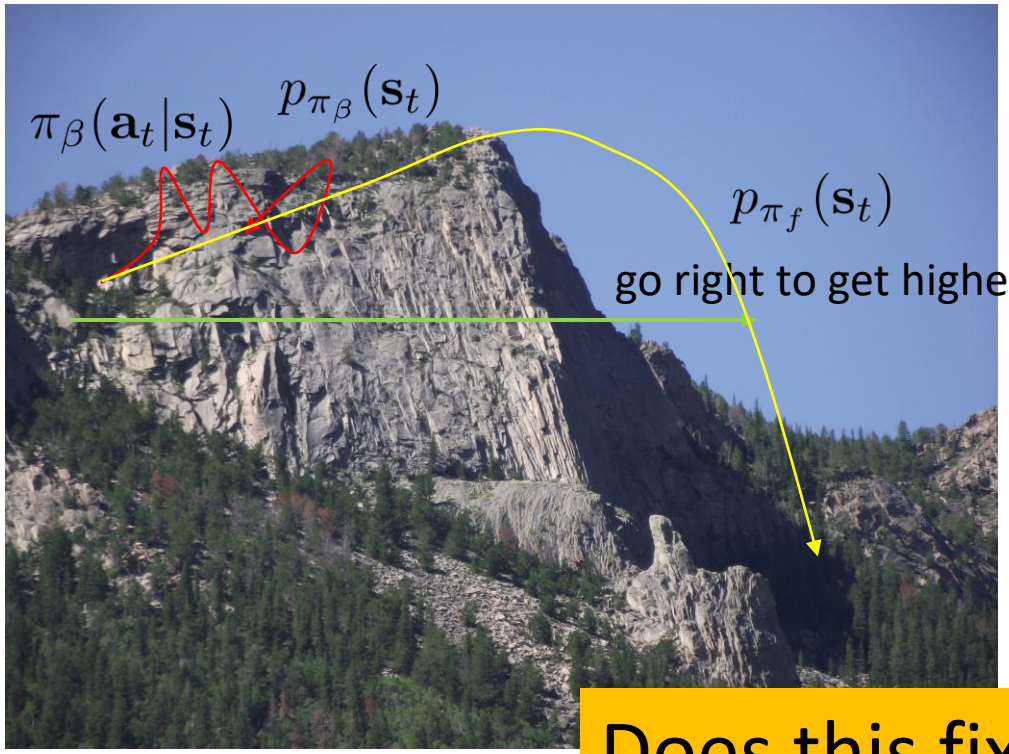
Fun to talk about in industry, because we can make lots of GPUs go brrr

Controls/RL problem

Often not fun to talk about, because planning, control, and RL are hard

Part 2:
Distributional shift and uncertainty

Distributional shift in model-based RL



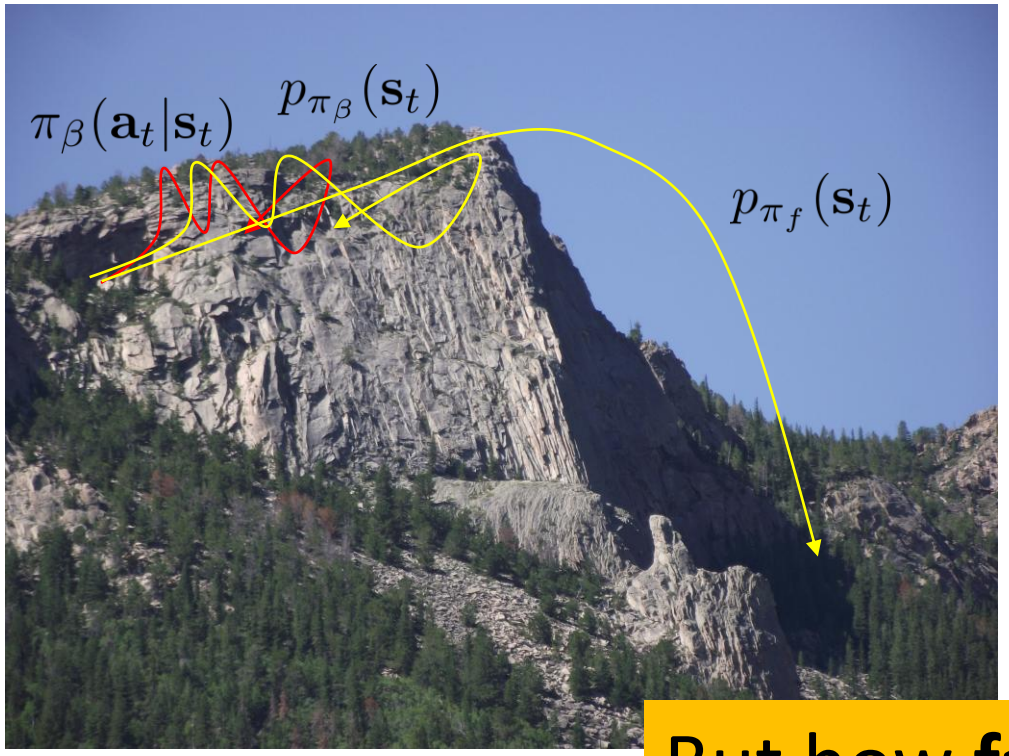
1. Get some data $\mathcal{D} = \{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i\}$ using π_β
2. Learn model $f(\mathbf{s}, \mathbf{a}) = \mathbf{s}'$
3. Learn optimal policy π_f under f
4. Set $\pi_\beta \leftarrow \pi_f$

$$p_{\pi_f}(\mathbf{s}_t) \neq p_{\pi_\beta}(\mathbf{s}_t)$$

Does this fix the problem?

Sort of…

Distributional shift in model-based RL



1. Get some data $\mathcal{D} = \{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i\}$ using π_β
2. Learn model $f(\mathbf{s}, \mathbf{a}) = \mathbf{s}'$
3. ~~Learn optimal policy π_f under f~~ improve the policy a bit under f?
4. Set $\pi_\beta \leftarrow \pi_f$

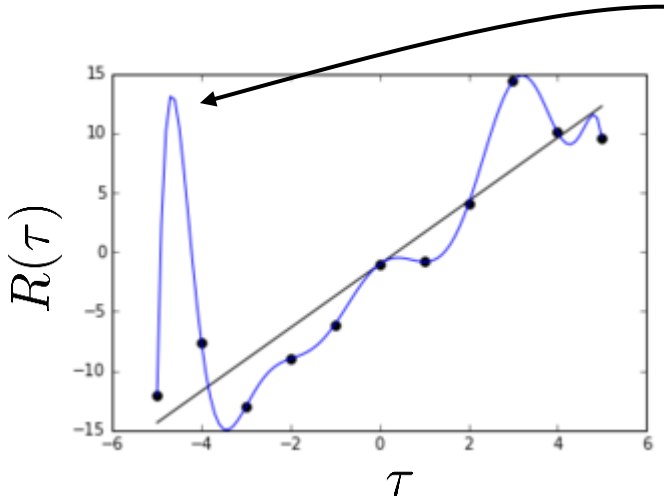
$$p_{\pi_f}(\mathbf{s}_t) \neq p_{\pi_\beta}(\mathbf{s}_t)$$

But how **far** do we go?

If we want to learn **fast**, then go as far as possible…

The problem is pretty bad!

1. Get some data $\mathcal{D} = \{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i\}$ using π_β
2. Learn model $f(\mathbf{s}, \mathbf{a}) = \mathbf{s}'$
3. ~~Learn optimal policy π_f under f~~ improve the policy a bit under f?
4. Set $\pi_\beta \leftarrow \pi_f$



very tempting to go here…

Distributional shift in model-based RL

1. Get some data $\mathcal{D} = \{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i\}$ using π_β
2. Learn model $f(\mathbf{s}, \mathbf{a}) = \mathbf{s}'$
3. Improve the policy "a bit" under f
3. Set $\pi_\beta \leftarrow \pi_f$

1. Get some data $\mathcal{D} = \{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i\}$ using π_β
2. Learn **probabilistic** model $p(\mathbf{s}'|\mathbf{s}, \mathbf{a})$
3. Get best policy π_p **in expectation** under p
3. Set $\pi_\beta \leftarrow \pi_p$

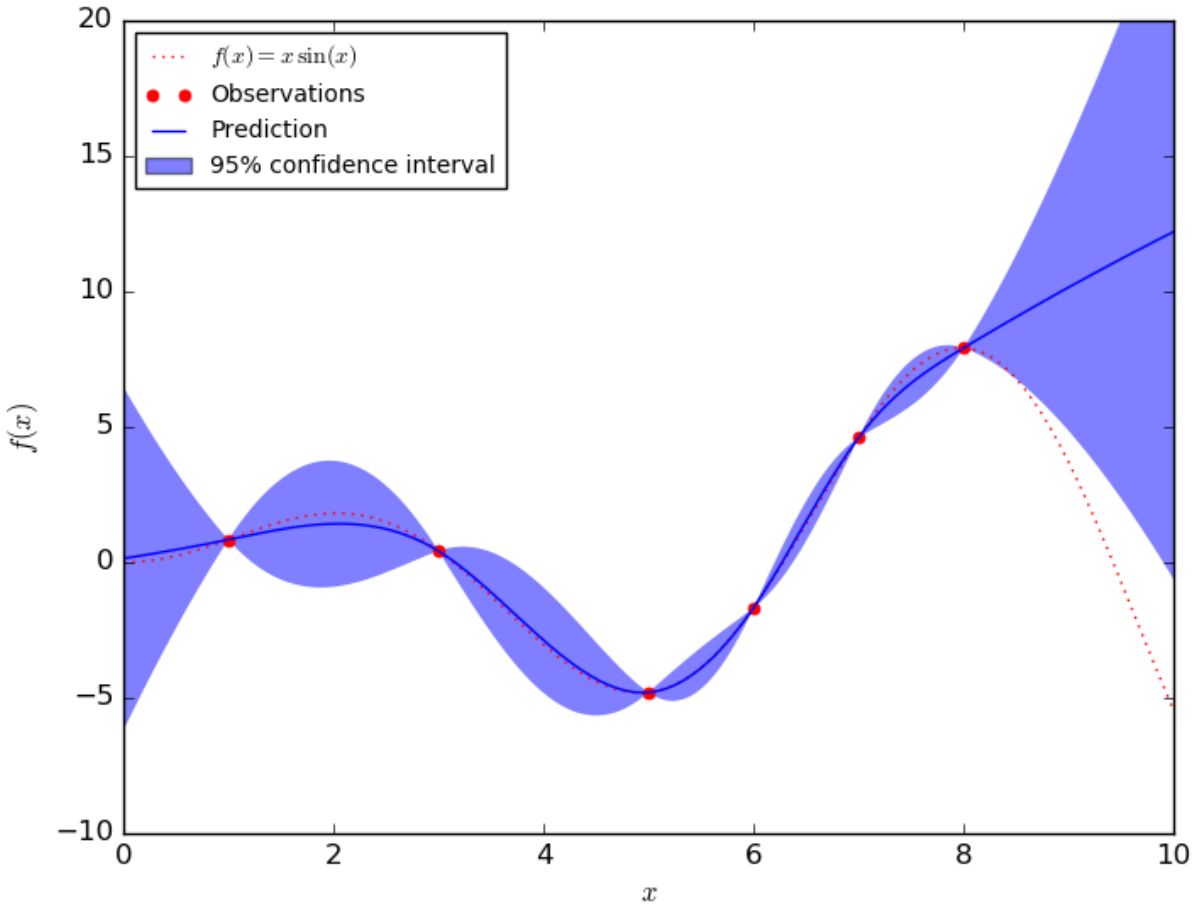
1. don't change the policy too much

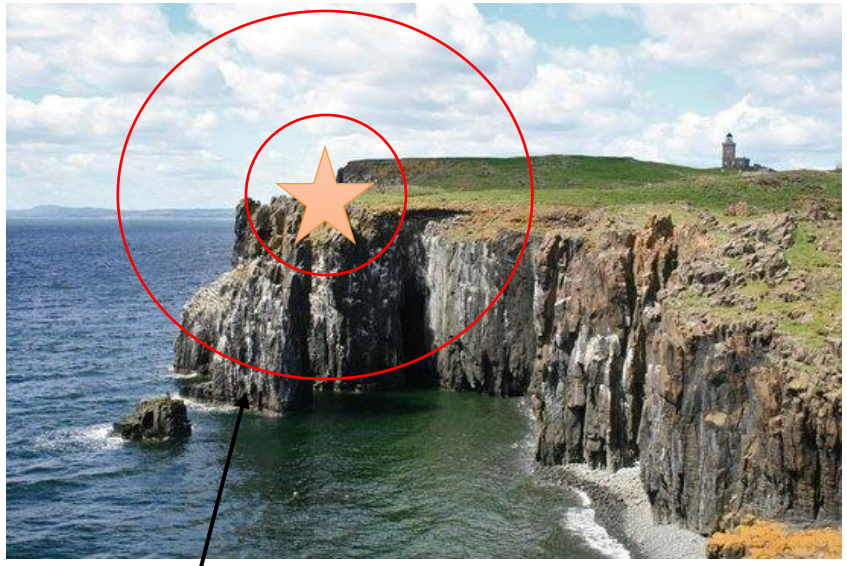
$D_{\mathrm{KL}}(\pi_f \| \pi_\beta) \leq \epsilon$ e.g., "trust region"

2. stick to places where the model is "confident"

- Use probabilistic, uncertainty-aware model
- Use penalty ("pessimism") in the face of model uncertainty

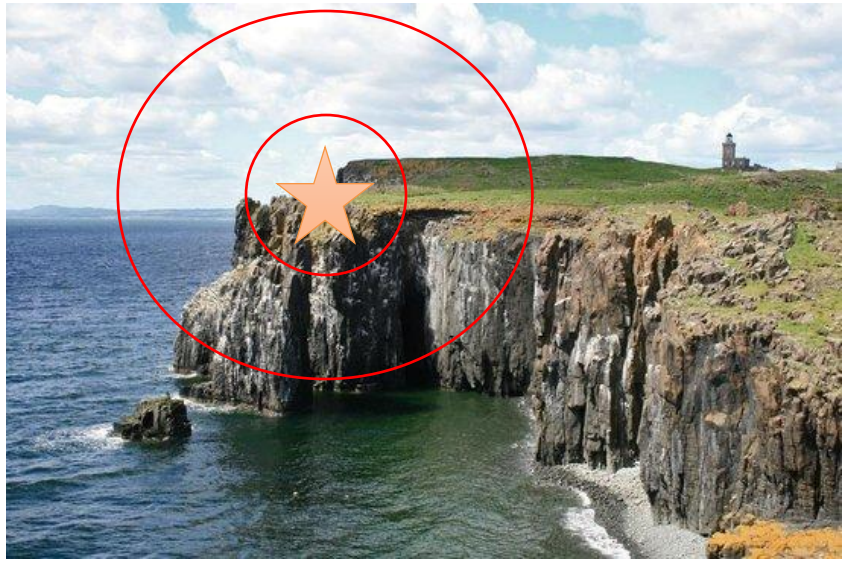
How can uncertainty estimation help?





expected reward under high-variance prediction
is **very** low, even though mean is the same!

There are a few caveats…



Need to explore to get better

Expected value is not the same as pessimistic value

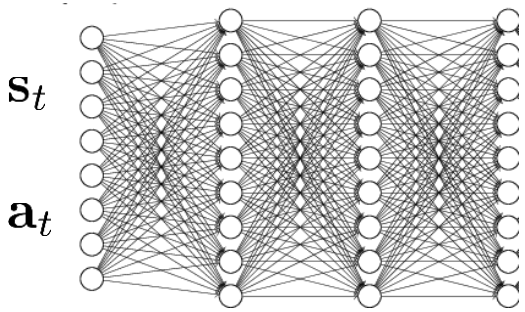
Expected value is not the same as optimistic value

…but expected value is often a good start

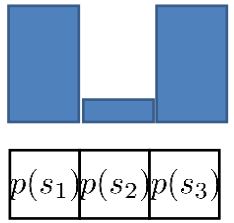
Part 3:
Uncertainty-aware neural networks

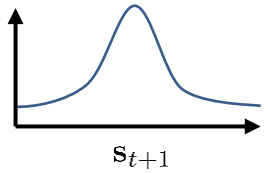
How can we have uncertainty-aware models?

Idea 1: use output entropy

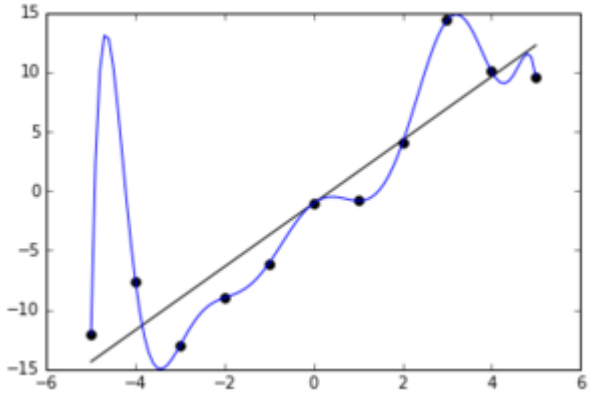


$$p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$$





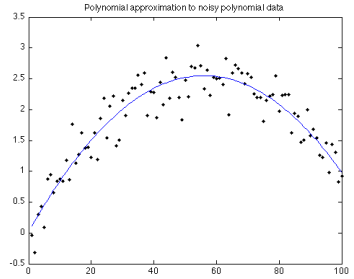
why is this not enough?



what is the variance here?

Two types of uncertainty:

aleatoric or *statistical* uncertainty →



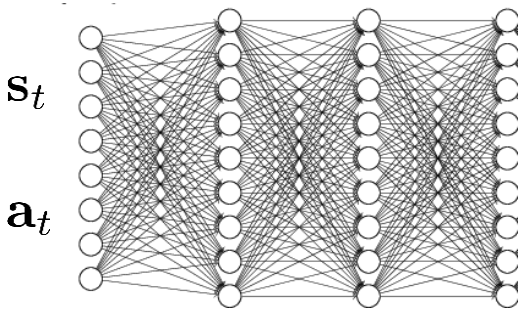
← *epistemic* or *model* uncertainty

"the model is certain about the data, but we are not certain about the model"

How can we have uncertainty-aware models?

Idea 2: estimate model uncertainty

"the model is certain about the data, but we are not certain about the model"



$p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$

parameters θ

usually, we estimate

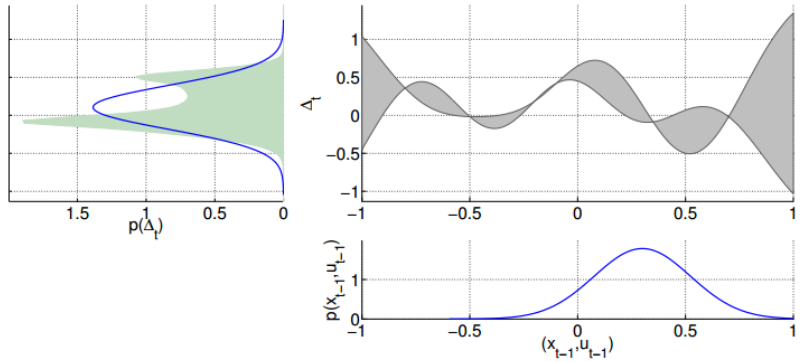
$$\arg\max_\theta \log p(\theta|\mathcal{D}) = \arg\max_\theta \log p(\mathcal{D}|\theta)$$

can we instead estimate $p(\theta|\mathcal{D})$?

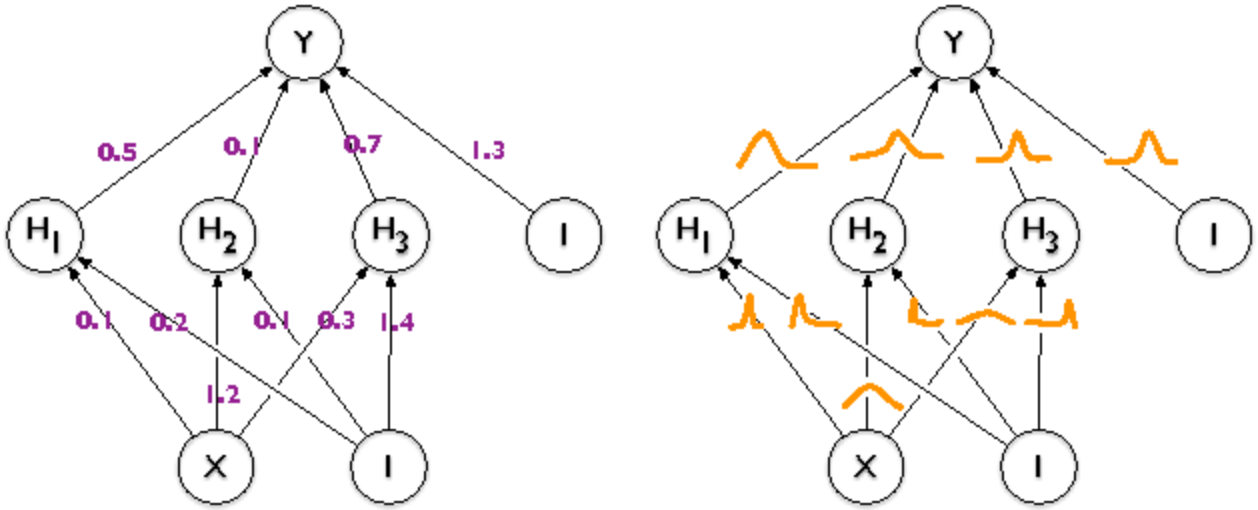
the entropy of this tells us the model uncertainty!

predict according to:

$$\int p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t, \theta) p(\theta|\mathcal{D}) d\theta$$



Quick overview of Bayesian neural networks



common approximation:

$$p(\theta|\mathcal{D}) = \prod_i p(\theta_i|\mathcal{D})$$

$$p(\theta_i|\mathcal{D}) = \mathcal{N}(\mu_i, \sigma_i)$$

expected weight

uncertainty about the weight

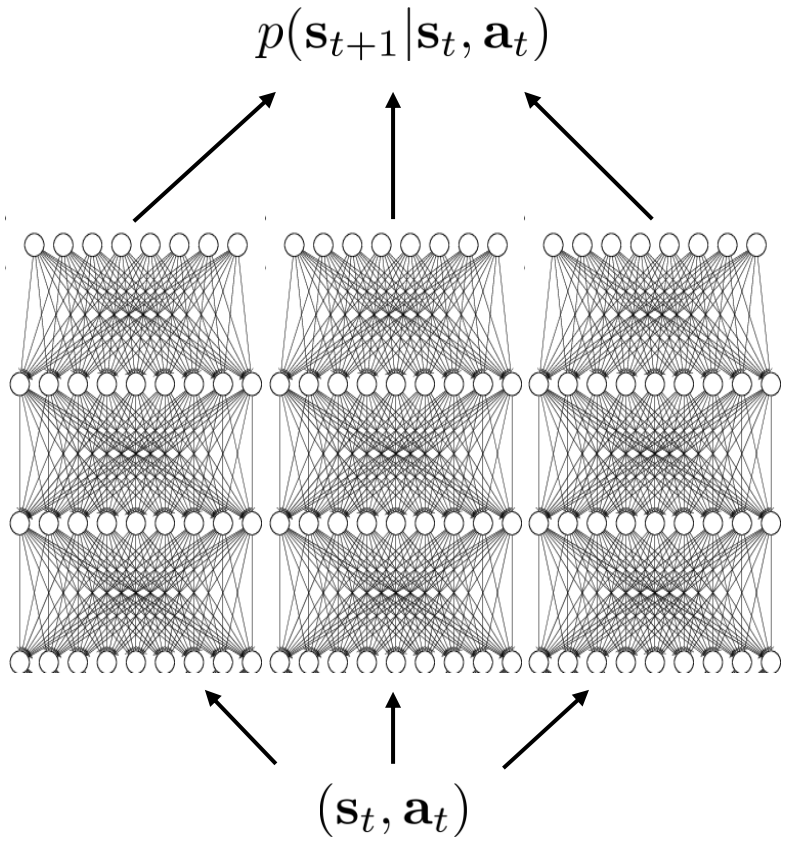
For more, see:

Blundell et al., Weight Uncertainty in Neural Networks

Gal et al., Concrete Dropout

We'll learn more about variational inference later!

Bootstrap ensembles



Train multiple models and see if they agree!

$$\text{formally:} \quad p(\theta|\mathcal{D}) \approx \frac{1}{N}\sum_i \delta(\theta_i)$$

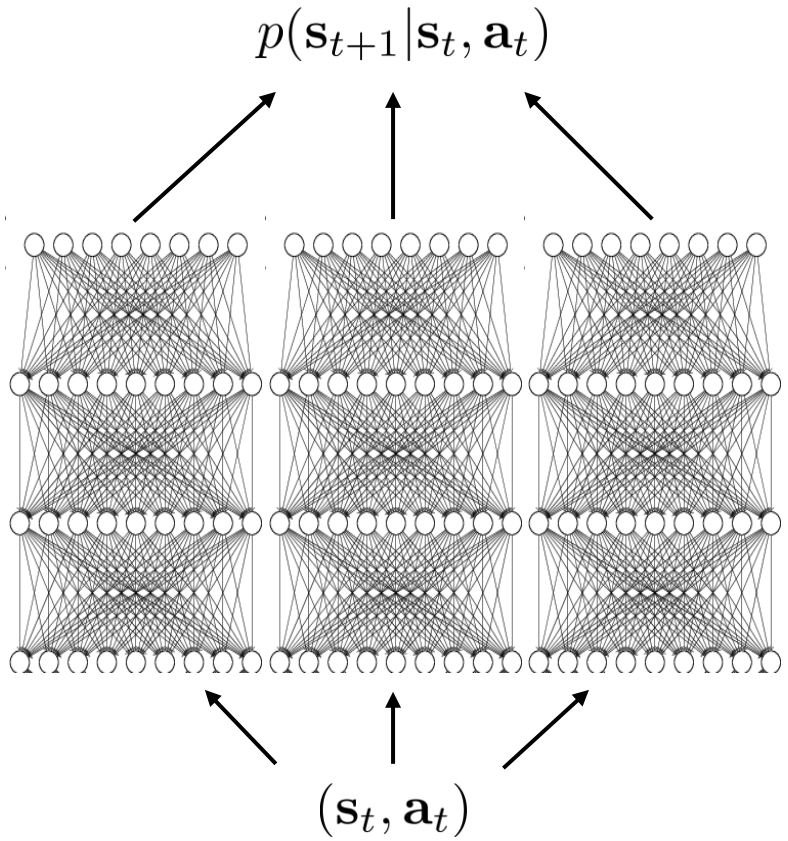
$$\int p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t, \theta)p(\theta|\mathcal{D})d\theta \approx \frac{1}{N}\sum_i p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t, \theta_i)$$

How to train?

Main idea: need to generate "independent" datasets to get "independent" models

θ_i is trained on $\mathcal{D}_i$, sampled *with replacement* from $\mathcal{D}$

Bootstrap ensembles in deep learning



This basically works

Very crude approximation, because the number of models is usually small (< 10)

Resampling with replacement is usually unnecessary, because SGD and random initialization usually makes the models sufficiently independent

Next time…

1. Get some data $\mathcal{D} = \{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i\}$
   collected using base policy $\pi_\beta(\mathbf{a}|\mathbf{s})$

2. Learn "simulator" $f(\mathbf{s}, \mathbf{a}) = \mathbf{s}'$

   $\min_f \sum_i ||f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i||^2$

use uncertainty-aware model to deal with distributional shift

3. Run your favorite RL method inside "simulator" f

could run one of the algorithms we learned about, but there are better options here